Lista de Exercícios para Revisão – P1
Estrutura de Dados 2
Profª Juliana

Questões abertas.

1. A partir do vetor de entrada V (8, 5, 3, 2, 1, 7) faça a ordenação utilizando o método *Bolha*. Demonstre passo a passo as verificações e trocas que ocorrem. Reescreva o vetor a cada troca. Qual o número de comparações que foram realizadas? E de trocas? Explicite qual é o fundamento do método, e a sua complexidade no pior, médio e melhor caso.

8	5	3	2	1	7
0	1	2	3	4	5

5	8	3	2	1	7
0	1	2	3	4	5

5	3	8	2	1	7
0	1	2	3	4	5

5	3	2	8	1	7
0	1	2	3	4	5

5	3	2	1	8	7
0	1	2	3	4	5

5	3	2	1	7	8
0	1	2	3	4	5

Tamanho = 6, t = tamanho -1
1ª Rodada: t = 5;
i=0; i<t? Sim
V[i]>V[i+1]? Sim (troca)
aux= V[i]
V[i]=V[i+1]
V[i+1] = aux
Atualiza índice (i++)

i=1, t=5, i<t? Sim
V[1]>V[2]? Sim (troca)
aux= V[1]
V[1]=V[2]
V[2] = aux
i++
....
i= 4, t=5, i<t? Sim
V[4]>V[5]? Sim (troca)
aux= V[4]
V[4]=V[5]
V[5] = aux
i++

Fundamento do Método: percorrer o vetor, comparar elementos adjacentes e fazer a troca dos elementos quando pertinente. O maior elemento flutua e é levado ao final do vetor. A cada rodada, diminui-se de 1 o tamanho do vetor que deverá ser percorrido, e retoma o processo da primeira posição do vetor.

A complexidade no melhor caso é O(n), quando o vetor já está ordenado, e o médio e pior caso é $O(n^2)$.

i<t? Não, então 1ª rodada termina.
Decrementa o valor de t, t=4;
O maior valor já está na posição certa (final do vetor).

5	3	2	1	7	8
0	1	2	3	4	5

3	5	2	1	7	8
0	1	2	3	4	5

3	2	5	1	7	8
0	1	2	3	4	5

3	2	1	5	7	8
0	1	2	3	4	5

2ª Rodada: t=4
i=0; i<t? Sim
V[i]>V[i+1]? Sim (troca)
aux= V[i]
V[i]=V[i+1]
V[i+1] = aux
Atualiza índice (i++)

....
i= 3, i<t? Sim
V[3]>V[34]? Não
Não troca e atualiza o i.

i<t? Não, então 2ª rodada termina.
Decrementa o valor de t, t=3;
O segundo maior valor já está na posição certa (final do vetor).

3	2	1	5	7	8
0	1	2	3	4	5

3	2	1	5	7	8
0	1	2	3	4	5

2	3	1	5	7	8
0	1	2	3	4	5

3ª Rodada: t = 3;
I=0, i<t? Sim
V[0]>V[1]? Sim (troca)
Atualiza índice (i++)

i=1; t = 3; i<t? Sim
V[1]>V[2]? Sim (troca)
Atualiza índice (i++)

2	1	3	5	7	8
0	1	2	3	4	5

V[2]>V[3]? Não
Não troca e atualiza o i.

i<t? Não, então 3ª rodada termina.
Decrementa o valor de t, t=2;
O terceiro maior valor já está na posição certa (final do vetor).

2	1	3	5	7	8
0	1	2	3	4	5

2	1	3	5	7	8
0	1	2	3	4	5

4ª Rodada: t = 2;
i=0 ,i<t? Sim
V[0]>V[1]? Sim (troca)
Atualiza índice (i++)
V[1]>V[2]? Não
Não troca e atualiza i

i<t? Não, então 4ª rodada termina.
Decrementa o valor de t, t=1;
O quarto maior valor já está na posição certa (final do vetor).

1	2	3	5	7	8
0	1	2	3	4	5

1	2	3	5	7	8
0	1	2	3	4	5

5ª Rodada: t = 1;
i=0, i<t? Sim
V[0]>V[1]? Não
Não troca e atualiza i

i<t? Não, então 5ª rodada termina.
Decrementa o valor de t, t=0;
O quinto maior valor já está na posição certa.

1	2	3	5	7	8
0	1	2	3	4	5

1	2	3	5	7	8
0	1	2	3	4	5

6ª rodada: t=0.
I=0, i<t ? => não, finaliza

Vetor Ordenado

2. A partir do vetor de entrada V (8, 5, 3, 2, 1, 7) faça a ordenação utilizando o método *Selection Sort*. Demonstre passo a passo as verificações e trocas que ocorrem. Reescreva o vetor a cada troca. Qual o número de comparações que foram realizadas? E de trocas? Explicite qual é o fundamento do método, e a sua complexidade no pior, médio e melhor caso.

Fundamento do Método: O algoritmo percorre o vetor e, a cada iteração, encontra o menor elemento no subvetor não ordenado e o coloca na posição correta.

Se o menor elemento encontrado não estiver na posição esperada, ocorre uma troca para colocá-lo na posição correta.

A complexidade no melhor caso é O(n), quando o vetor já está ordenado, e o médio e pior caso é $O(n^2)$.

i=0

8	5	3	2	1	7
0	1	2	3	4	5

Aux=0 → posição

J=1

8	5	3	2	1	7
0	1	2	3	4	5

Aux=1 → posição

J=2

8	5	3	2	1	7
0	1	2	3	4	5

Aux=2 → posição

J=3

8	5	3	2	1	7
0	1	2	3	4	5

Aux=3 → posição

J=4

8	5	3	2	1	7
0	1	2	3	4	5

Aux=4 → posição

J=5

8	5	3	2	1	7
0	1	2	3	4	5

Aux=4 → posição

J=6

8	5	3	2	1	7
0	1	2	3	4	5

1	5	3	2	8	7
0	1	2	3	4	5

1ª Rodada: tam = 6, i =0
i<tam-1? Sim
aux = i; aux=0

j=i+1; => j=1
j<tam? Sim
V[aux]>V[j]? Sim (atualiza aux)
Aux=j; => aux =1
Atualiza j. => j=1+1=2

j=2
j<tam? Sim
V[1]>V[2]? Sim (atualiza aux)
aux=j; => aux =2
Atualiza j. => j=2+1=3

j=3
j<tam? Sim
V[aux]>V[j]? Sim (atualiza aux)
aux=j; => aux =3
Atualiza j. => j=3+1=4

j=4
j<tam? Sim
V[3]>V[4]? Sim (atualiza aux)
aux=j; => aux =4
Atualiza j. => j=4+1=5

j=5
j<tam? Sim
V[4]>V[5]? Não (aux fica igual)
Atualiza j. => j=5+1=6

j=6
j<tam? Não
Troca o valor V[aux] com V[i]
Chegou ao fim do vetor, finaliza a 1ª rodada

i=1

1	5	3	2	8	7
0	1	2	3	4	5

Aux=1 → posição

J=2

1	5	3	2	8	7
0	1	2	3	4	5

Aux=2 → posição

J=3

1	5	3	2	8	7
0	1	2	3	4	5

Aux=3 → posição

J=4

1	5	3	2	8	7
0	1	2	3	4	5

Aux=3 → posição

J=5

1	5	3	2	8	7
0	1	2	3	4	5

Aux=3 → posição

J=6

1	5	3	2	8	7
0	1	2	3	4	5

1	2	3	5	8	7
0	1	2	3	4	5

2ª Rodada: tam = 6, i =1
i<tam-1? Sim
aux = i; aux=1

j=i+1; => j=1+1=2
j<tam? Sim
V[aux]>V[j]? Sim (atualiza aux)
Aux=j; => aux =2
Atualiza j. => j=2+1=3

j=3
j<tam? Sim
V[2]>V[3]? Sim (atualiza aux)
aux=j; => aux =3
Atualiza j. => j=3+1=4

j=4, aux =3
j<tam? Sim
V[3]>V[4]? Não (aux fica igual)
Atualiza j. => j=4+1=5

j=5, aux=3
j<tam? Sim
V[3]>V[5]? Não (aux fica igual)
Atualiza j. => j=5+1=6

j=6, aux=3
j<tam? Não
Troca o valor V[aux] com V[i]
Chegou ao fim do vetor, finaliza a 2ª rodada

i=2 => j=i+1

1	2	3	5	8	7
0	1	2	3	4	5

Aux=2 → posição

J= 3

1	2	3	5	8	7
0	1	2	3	4	5

Aux=2

J=4

1	2	3	5	8	7
0	1	2	3	4	5

Aux=2

J=5

1	2	3	5	8	7
0	1	2	3	4	5

Aux = 2

J=6

1	2	3	5	8	7
0	1	2	3	4	5

1	2	3	5	8	7
0	1	2	3	4	5

3ª Rodada: tam = 6, i =2
i<tam-1? Sim
aux = i; aux=2

j=3
j<tam? Sim
V[2]>V[3]? Não (aux fica igual) continua aux =2
Atualiza j => j=3+1=4

j=4
j<tam? Sim
V[2]>V[4]? Não (aux fica igual) continua aux =2
Atualiza j. => j=4+1=5

j=5
j<tam? Sim
V[2]>V[5]? Não (aux fica igual) continua aux =2
Atualiza j. => j=5+1=6

j=6
j<tam? Não
Troca o valor V[aux] com V[i]
Chegou ao fim do vetor, finaliza a 3ª rodada

i=3

1	2	3	5	8	7
0	1	2	3	4	5

Aux=3 → posição

J=4

1	2	3	5	8	7
0	1	2	3	4	5

Aux=3 → posição

J=5

1	2	3	5	8	7
0	1	2	3	4	5

Aux=3 → posição

J=6

1	2	3	5	8	7
0	1	2	3	4	5

4ª Rodada: tam = 6, i =3
i<tam-1? Sim
aux = i; aux=3

j=4, aux =3
j<tam? Sim
V[3]>V[4]? Não (aux fica igual)
Atualiza j. => j=4+1=5

j=5, aux=3
j<tam? Sim
V[3]>V[5]? Não (aux fica igual)
Atualiza j. => j=5+1=6

j=6, aux=3
j<tam? Não
Troca o valor V[aux] com V[i]
Chegou ao fim do vetor, finaliza a 4ª rodada

i=4

1	2	3	5	8	7
0	1	2	3	4	5

Aux=4 → posição

J=5

1	2	3	5	8	7
0	1	2	3	4	5

Aux=5 → posição

J=6

1	2	3	5	8	7
0	1	2	3	4	5

1	2	3	5	7	8
0	1	2	3	4	5

5ª Rodada: tam = 6, i =4
i<tam-1? Sim
aux = i; aux=4

j=5, aux=4
j<tam? Sim
V[4]>V[5]? Sim (atualiza aux)
Aux = j => aux = 5
Atualiza j. => j=5+1=6

j=6, aux=3
j<tam? Não
Troca o valor V[aux] com V[i]
Chegou ao fim do vetor, finaliza a 4ª rodada

i=5 => i<tam? Falso.

1	2	3	5	7	8
0	1	2	3	4	5

6ª rodada: i=5.
I<tam? Não => então

Vetor Ordenado

3. A partir do vetor de entrada V (8, 5, 3, 2, 1, 7) faça a ordenação utilizando o método *Insertion Sort*. Demonstre passo a passo as verificações e trocas que ocorrem. Reescreva o vetor a cada troca. Qual o número de comparações que foram realizadas? E de trocas? Explicite qual é o fundamento do método, e a sua complexidade no pior, médio e melhor caso.

8	5	3	2	1	7
0	1	2	3	4	5

Resumo de como funciona o método: O algoritmo admite que o primeiro elemento do vetor já está ordenado, como se fosse um sub vetor ordenado. Cria uma variável aux e atribui o valor do segundo elemento do vetor. Compara essa com o último elemento desse sub vetor ordenado, se o aux for menor, o valor desse sub vetor é copiado (duplicado) para o seu adjacente, essa comparação é feita até chegar na posição 0 (sub vetor ordenado ou até encontrar um valor que seja menor que o aux nesse sub vetor). Quanto essa verificação finalizar o valor que está no aux é inserido na posição correta. O próximo elemento do vetor (adjacente ao vetor ordenado) é escolhido como aux e todo o processo é iniciado novamente (compara aux com os elementos do sub vetor ordenado). Quando se chegar no último elemento do vetor, faz a comparação e após a inserção o vetor já está ordenado.

8	5	3	2	1	7
0	1	2	3	4	5

Aux = 5

8	8	3	2	1	7
0	1	2	3	4	5

Aux = 5

5	8	3	2	1	7
0	1	2	3	4	5

5	8	3	2	1	7
0	1	2	3	4	5

Aux = 3

5	8	8	2	1	7
0	1	2	3	4	5

Aux = 3

5	5	8	2	1	7
0	1	2	3	4	5

3	5	8	2	1	7
0	1	2	3	4	5

3	5	8	2	1	7
0	1	2	3	4	5

Aux = 2

3	5	8	2	1	7
0	1	2	3	4	5

Aux = 2

3	5	8	8	1	7
0	1	2	3	4	5

Aux = 2

3	5	5	8	1	7
0	1	2	3	4	5

Aux = 2

3	5	5	8	1	7
0	1	2	3	4	5

Aux = 2

3	3	5	8	1	7
0	1	2	3	4	5

2	3	5	8	1	7
0	1	2	3	4	5

2	3	5	8	1	7
0	1	2	3	4	5

Aux = 1

2	3	5	8	1	7
0	1	2	3	4	5

Aux = 1

2	3	5	8	8	7
0	1	2	3	4	5

Aux = 1

2	3	5	5	8	7
0	1	2	3	4	5

Aux = 1

2	3	5	5	8	7
0	1	2	3	4	5

Aux = 1

2	2	3	5	8	7
0	1	2	3	4	5

Aux = 1

1	2	3	5	8	7
0	1	2	3	4	5

1	2	3	5	8	7
0	1	2	3	4	5

Aux = 7

1	2	3	5	8	7
0	1	2	3	4	5

Aux = 7

1	2	3	5	8	8
0	1	2	3	4	5

Aux = 7

2	3	3	5	7	8
0	1	2	3	4	5

4. A partir do vetor de entrada V (8, 5, 11, 2, 1, 7, 9, 3, 10, 6, 4) faça a ordenação utilizando o método *Merge Sort*. Demonstre passo a passo as divisões, verificações e trocas que ocorrem. Reescreva o vetor a cada divisão. Demonstre os passos de forma recursiva. Explicite qual é o fundamento do método, e a sua complexidade no pior, médio e melhor caso.

MergeSort(V,0,10){ // PRIMEIRA chamada V=(8, 5, 11, 2, 1, 7, 9, 3, 10, 6, 4)

```
I =0, F= 10, Meio = 0 + (10-0)/2 → Meio = 5;
MergeSort(V, 0,5){ // ch1, lado esquerdo V=(8, 5, 11, 2, 1, 7)
    I=0, f=5;
    Meio = 0+(5-0)/2 -> Meio = 2
    MergeSort(V, 0, 2){ //ch 2, lado esquerdo chamada V=(8, 5, 11)
        I = 0, F =2;
        Meio = 1
        MergeSort(V,0,1) {  ch3, lado esquerdo V= (8,5)
            I=0, F=1;
            Meio = 0 + (1-0)/2 → Meio = 0;
            MergeSort(V, 0,0){ //ch4, lado esquerdo:V={2}
                Return;}
            MergeSort(V, 1,1){ //ch4, lado direito:V={1}
                Return;}
            Merge (V, 0, 0, 1) { } => V (1,2) //ch4, ordena
            }
        }
        MergeSort(V,2,2) {  ch3, lado direito V= (11)
            Return;
        }
        Merge(V, 0,1,2) { }ch3, ordena V=> 5,8,11
    }
    MergeSort(V, 3, 5){ //ch 2, lado direito chamada V=(2, 1, 7)
        I = 3, F =5;
        Meio = 4;
        MergeSort(V,3,4) {  ch3, lado esquerdo V= (2,1)


        MergeSort(V,5,5) {  ch3, lado direito V= (7)
            Return;}
        Merge (V, 3,4,5) {} ch3, ordena V=> (1,2,7)

    }
    Merge (V, 0,2,5) {} ch2, ordena V=>(1,2,5,7,8,11)
}
```

```
MergeSort(V, 6,10){ // ch1, lado direito V=(9, 3, 10, 6, 4)

    I = 6, F=10;
    Meio =8;
    MergeSort(V, 6, 8) { // ch2, lado esquerdo V= (9, 3, 10)
            I =6, F=8;
            Meio = 7;
            MergeSort(V, 6,7){ ch3, lado esquerdo V(9,3)
                I=6, F=7;
                Meio = 6;
                MergeSort(V, 6, 6){ // ch4, lado esquerdo V(9)
                    Return;}
                MergeSort(V, 7,7) { //ch4, lado direito V(3)
                     Return; }
                Merge (V, 6,6,7){} //ch4, ordena V=>(3,9)
            }
            MergeSort(V,8,8){ ch3, lado direito V(10)
                 Return;
            Merge(V, 6,7,8) {} ){ ch3, ordena  V(3,9,10)
    }
    MergeSort(V,9,10) { //ch2, lado direito V = (6,4)
            I= 9, F=10;
            Meio = 9;
            MergeSort(V,9,9) { //ch3, lado esquerdo V(6)
                 Return;}
            MergeSort(V,10,10){ { //ch3, lado direito V(4)
                Return;}
            Merge(V, 9,9,10) {} // ch3, ordena V=>(4,6)
    }
    Merge(V, 6, 8, 10) { }//ch2, ordena   V=> (3,4,6,9,10)

}

Merge(V, 0,5,10){ }// ch1, ordena V=(1,2,4,5,6,7,8,9,10,11)
```

Escolher um dos Merge() para realizar o passo a passo da ordenação